Natural Language Processing
Lab 2

Lonny Chen: l.chen@hertie-school.org
GRAD-E1282, Fall 2026
9-10 September 2026

Today’s Agenda

1. Regex Warmup
2. Data Cleaning Activity
3. Rules-Based Tokenize
4. Normalization
5. Data-Driven Tokenize

Warmup 1

```
^(cat|dog)s?$
```

catdog

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

Warmup 1

```
^(cat|dog)s?$
```

catdog 

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

Warmup 1

```
^(cat|dog)s?$
```

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

`catdog` 

`dogs`

Warmup 1

```
^(cat|dog)s?$
```

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

`catdog` 

`dogs`

Warmup 1

```
^(cat|dog)s?$
```

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

catdog 

dogs

dogs are barking

Warmup 1

```
^(cat|dog)s?$
```

`^`	`(cat|dog)`	`s?`	`$`
start of string	either cat or dog	zero or one s	end of string

`catdog` 

`dogs`

`dogs are barking` 

Warmup 2

```
0[1-9]|1[012]
```

00

Warmup 2

```
0[1-9]|1[012]
```

00 

Warmup 2

```
0[1-9]|1[012]
```

00 

20

Warmup 2

```
0[1-9]|1[012]
```

00 ❌

20 ❌

Warmup 2

```
0[1-9]|1[012]
```

00 ❌

20 ❌

11

Warmup 2

```
0[1-9]|1[012]
```

`0[1-9]`	`|`	`1[012]`
digits between 01-09	OR	digits between 10-12

`00` ❌

`20` ❌

`11`

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

Keep it simple!

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

Keep it simple!

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

```
Keep it simple!
```

```
This is just great
```

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

`Keep it simple!`

`This is just great` ❌

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

`Keep it simple!`

`This is just great` ❌

`!#%$*`

Warmup 3

```
^\S+(\s+\S+){0,2}$
```

`^\S+`

start string with non-whitespace

`(\s+\S+)`

followed by whitespace then non-whitespace

`{0,2}$`

zero, once, or twice only until string end

```
Keep it simple!
```

```
This is just great
```



```
!#%$*
```

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

```
from #HertieLoveeeee in
```

`#HertieL`	`[ou]+`	`v`	`e*`
exact start of substring	from the set of o and u once or more	just a v	an e zero or more times

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

```
from #HertieLoveeeeee in
```

`#HertieL`	`[ou]+`	`v`	`e*`
exact start of substring	from the set of o and u once or more	just a v	an e zero or more times

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

`#HertieL`

exact start of substring

`[ou]+`

from the set of o and u once or more

`v`

just a v

`e*`

an e zero or more times

```
from #HertieLoveeeeee in

$ome #HertieLuv with
```

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

`#HertieL`

exact start of substring

`[ou]+`

from the set of o and u once or more

`v`

just a v

`e*`

an e zero or more times

```
from #HertieLoveeeeee in

$ome #HertieLuv with
```

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

`#HertieL`	`[ou]+`	`v`	`e*`
exact start of substring	from the set of o and u once or more	just a v	an e zero or more times

```
from #HertieLoveeeeee in

$ome #HertieLuv with

I have #HertieLOuvee
```

Activity - Cleaning 1

```
r"#HertieL[ou]+ve*"
```

`#HertieL`	`[ou]+`	`v`	`e*`
exact start of substring	from the set of o and u once or more	just a v	an e zero or more times

```
from #HertieLoveeeee in

$ome #HertieLuv with

I have #HertieLOuvee
```



Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

M.D.S.

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

```
M.D.S.
```

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

```
M.D.S.
```

M-D-S

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

`M.D.S.`

M-D-S ✗

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

`M.D.S.`

`M-D-S` ✗

`MDdS`

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

Activity - Cleaning 2

```
r"[Mm]\.*[Dd]\.*[Ss]\.*"
```

`[Mm]`	`\.*`	`[Dd]`	`\.*`	`[Ss]`	`\.*`
case-folded m	period zero of more times	case-folded d	period zero of more times	case-folded s	period zero of more times

`M.D.S.`

`M-D-S` ✗

`MDdS` ✗

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

```
€2021
```

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

`€2021` 

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

`€2021` ✗

`of 2021`

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

`€2021` ✗

`of 2021`

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

€2021 ✗

of 2021

graduate 32021

Activity - Cleaning 3

```
r"[^€]202[1-2]"
```

`[^€]` not €

`202` exactly 202

`[1-2]` digits 1-2

€2021 ✗

of 2021

graduate 32021

Activity - Cleaning 4

```
r"([A-Z]\w+)\s+\1"
```

```
Nikki Nikki
```

`[A-Z]` first capital letter

`\w+` any alpha-numeric

`\s+` whitespace once or more

`\1` back-reference to capture group 1

`([A-Z]\w+)` save capture group 1

Activity - Cleaning 4

```
r"([A-Z]\w+)\s+\1"
```

```
Nikki Nikki
```

`[A-Z]` first capital letter

`\w+` any alpha-numeric

`\s+` whitespace once or more

`\1` back-reference to capture group 1

`([A-Z]\w+)` save capture group 1

Activity - Cleaning 4

```
r"([A-Z]\w+)\s+\1"
```

```
Nikki Nikki
```

```
Nikki Nikk
```

`[A-Z]` first capital letter

`\w+` any alpha-numeric

`\s+` whitespace once or more

`\1` back-reference to capture group 1

`([A-Z]\w+)` save capture group 1

Activity - Cleaning 4

```
r"([A-Z]\w+)\s+\1"
```

```
Nikki Nikki
```

```
Nikki Nikk
```



`[A-Z]` first capital letter

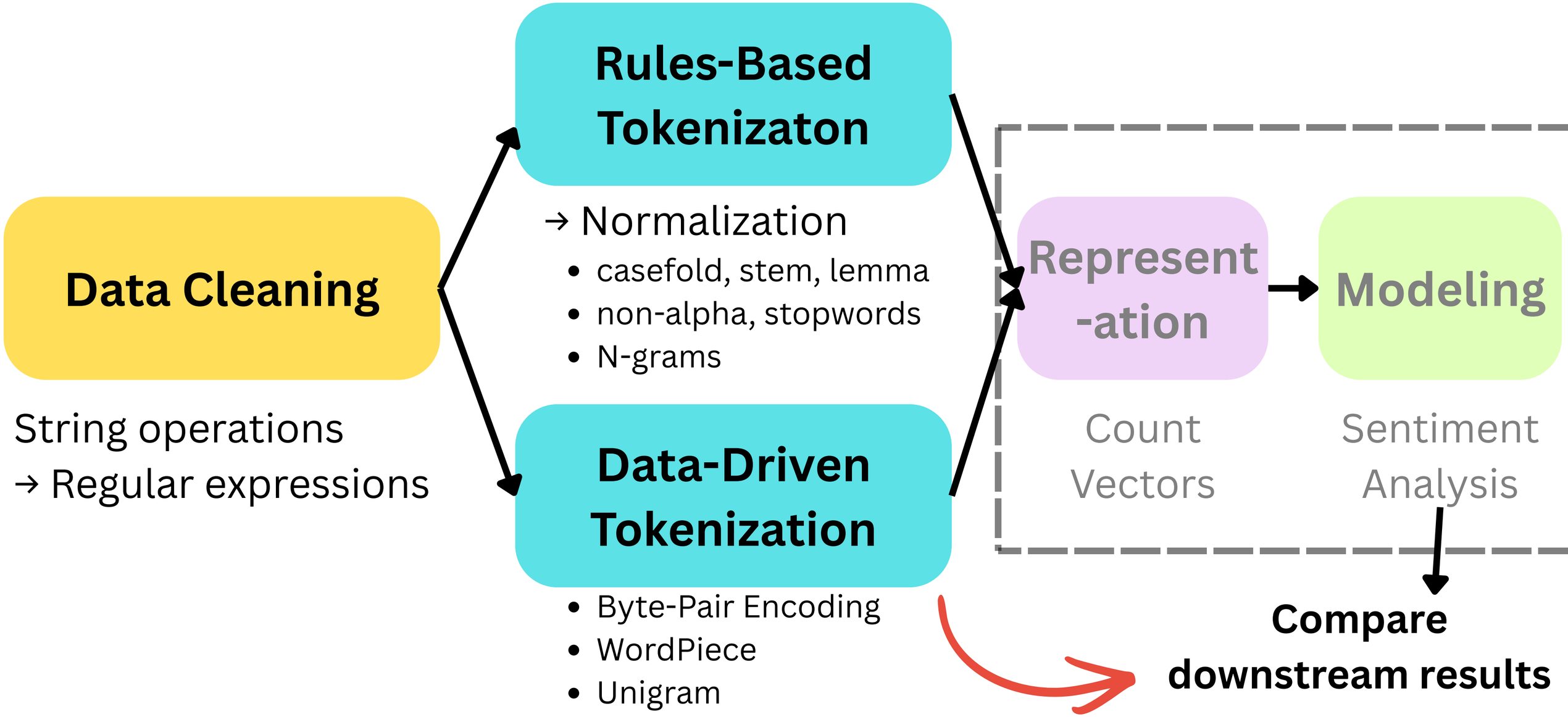
`\w+` any alpha-numeric

`\s+` whitespace once or more

`\1` back-reference to capture group 1

`([A-Z]\w+)` save capture group 1

Lab 2 Pipeline



Rules-Based Tokenize: Whitespace

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = text.split()
```

Rules-Based Tokenize: Whitespace

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = text.split()
```



```
['The', 'San', 'Francisco-based', 'restaurant', "doesn't", 'charge', '$10.']
```

Rules-Based Tokenize: Whitespace

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = text.split()
```



```
['The', 'San', 'Francisco-based', 'restaurant', "doesn't", 'charge', '$10.']
```







Rules-Based Tokenize: spaCy

"The San Francisco-based restaurant doesn't charge $10."

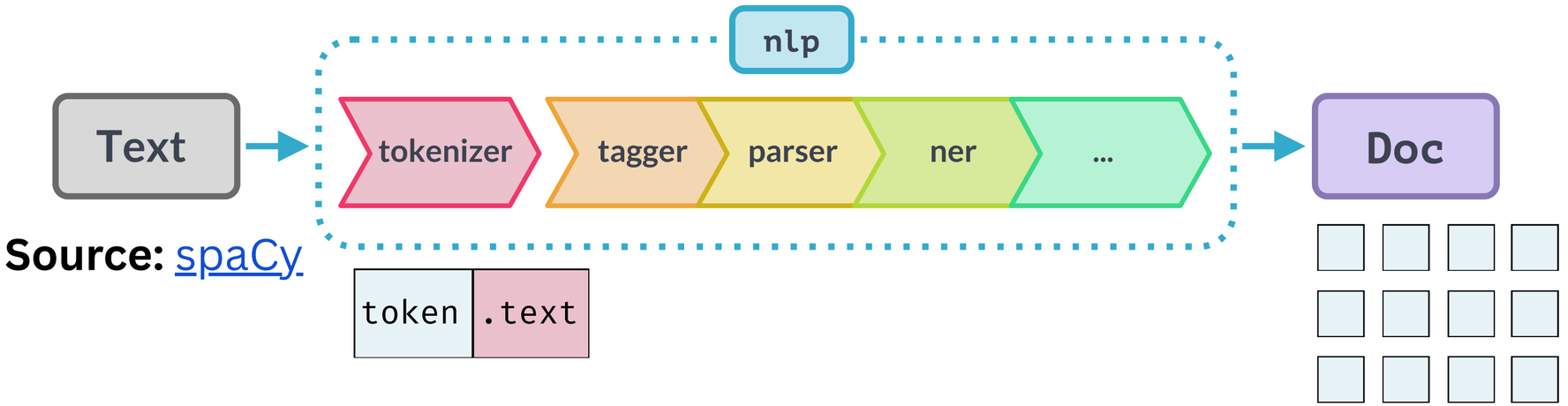


```
doc = nlp(text)
tokens = [token.text for token in doc]
```

Rules-Based Tokenize: spaCy

```
"The San Francisco-based restaurant doesn't charge $10."
```

```
doc = nlp(text)
tokens = [token.text for token in doc]
```



Source: spaCy

Rules-Based Tokenize: spaCy

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
doc = nlp(text)
tokens = [token.text for token in doc]
```



```
['The', 'San', 'Francisco', '-', 'based', 'restaurant', 'does', "n't",
                        'charge', '$', '10', '.']
```

Rules-Based Tokenize: spaCy

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
doc = nlp(text)
tokens = [token.text for token in doc]
```

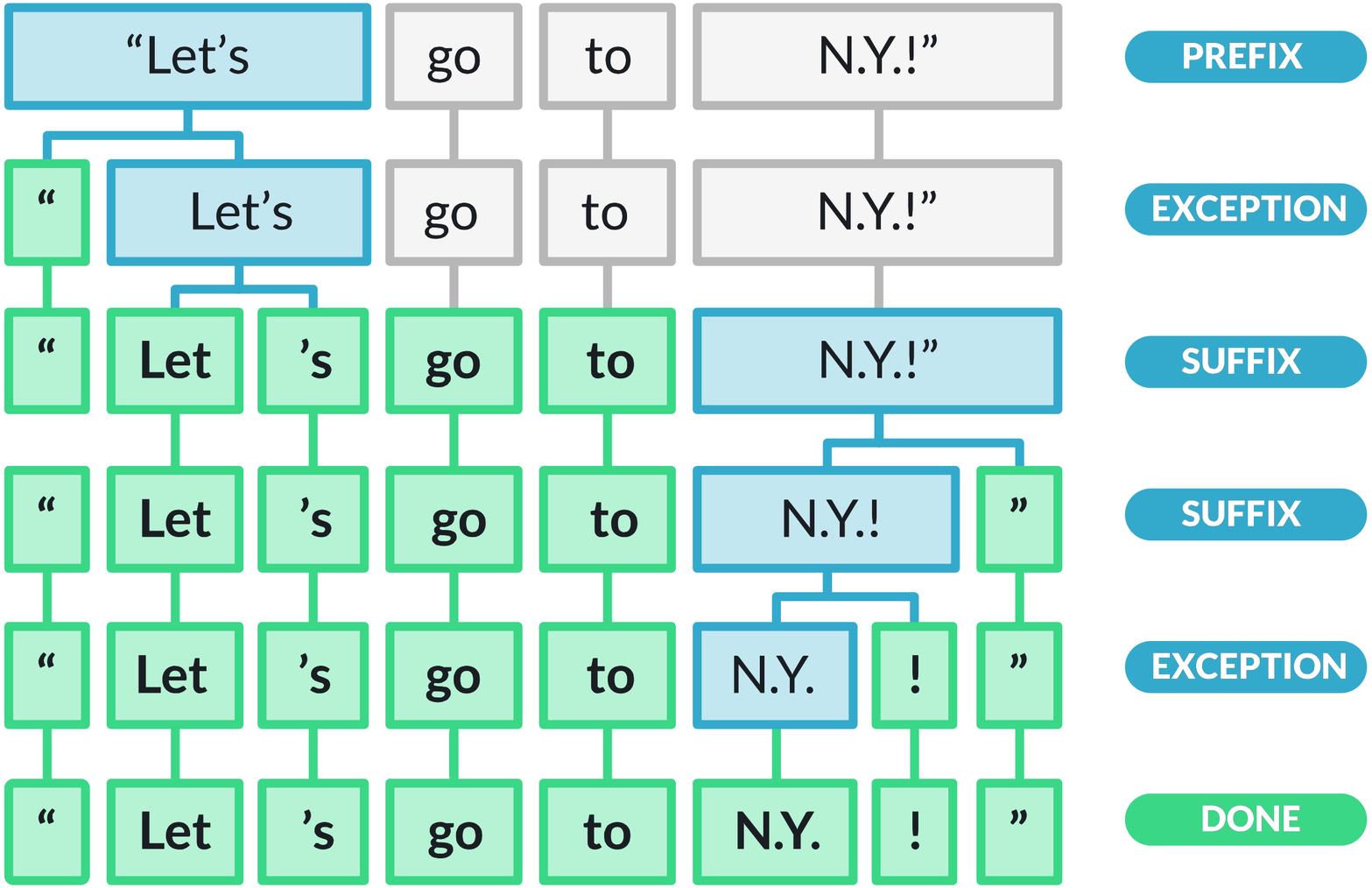


```
['The', 'San', 'Francisco', '-', 'based', 'restaurant', 'does', "n't",
                        'charge', '$', '10', '.']
```

```
docs_as_tokens = list(nlp.pipe(dft['text']))
[[token.text for token in doc] for doc in docs_as_tokens]
```

use **nlp.pipe** to run a collection/list/series of texts through the pipeline

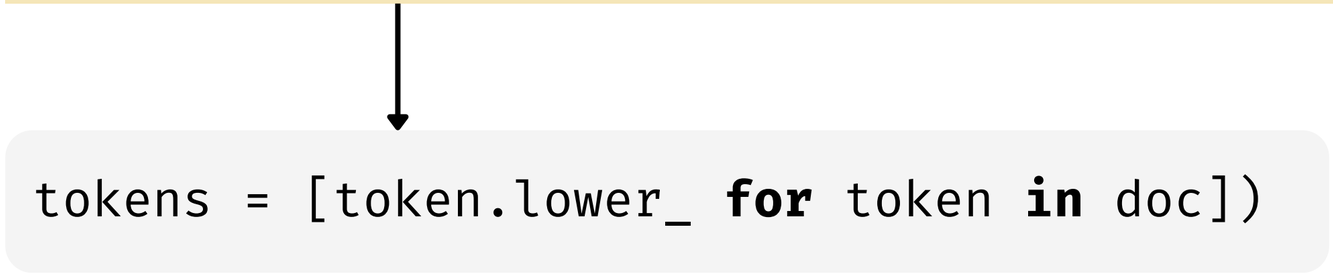
Rules-Based Tokenize: spaCy



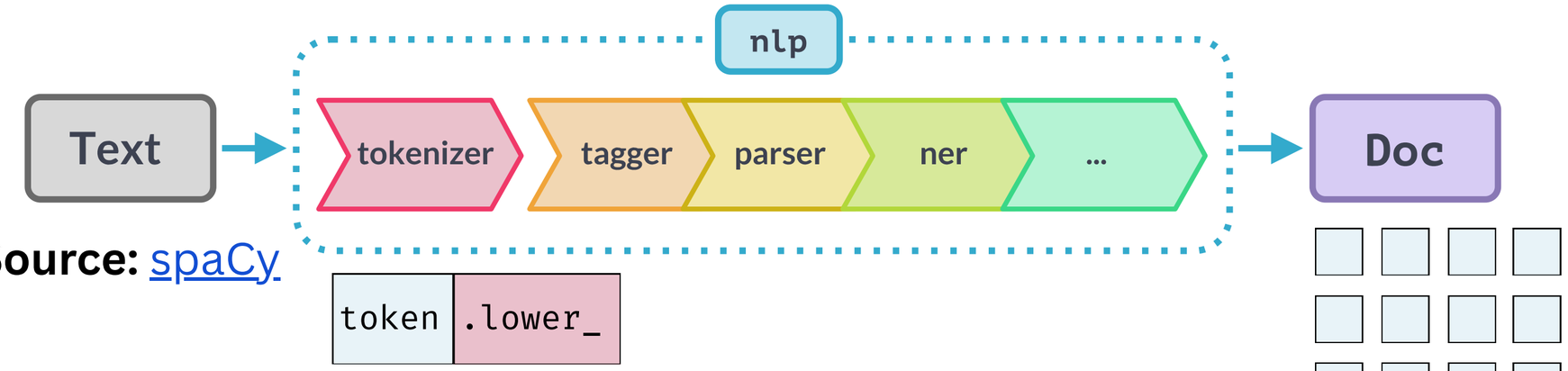
Source: spaCy

Normalization: Case Folding

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lower_ for token in doc])
```



Source: spaCy

Normalization: Case Folding

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lower_ for token in doc])
```



```
['the', 'san', 'francisco', '-', 'based', 'restaurant', 'does', "n't",
                        'charge', '$', '10', '.']
```

Normalization: Case Folding

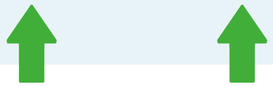
```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lower_ for token in doc])
```



```
['the', 'san', 'francisco', '-', 'based', 'restaurant', 'does', "n't",
                            'charge', '$', '10', '.']
```



Normalization: Stemming

"The San Francisco-based restaurant doesn't charge $10."



```
my_stemmer = SnowballStemmer('english') #NLTK
tokens = [my_stemmer.stem(token.text) for token in doc]
```

Normalization: Stemming

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
my_stemmer = SnowballStemmer('english') #NLTK
tokens = [my_stemmer.stem(token.text) for token in doc]
```



```
['the', 'san', 'francisco', '-', 'base', 'restaur', 'doe', "n't", 'charg',
                                '$', '10', '.']
```

Normalization: Stemming

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
my_stemmer = SnowballStemmer('english') #NLTK
tokens = [my_stemmer.stem(token.text) for token in doc]
```











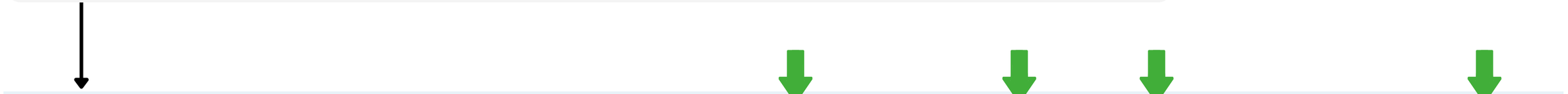
```
['the', 'san', 'francisco', '-', 'base', 'restaur', 'doe', "n't", 'charg',
                                '$', '10', '.']
```

Normalization: Stemming

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
my_stemmer = SnowballStemmer('english') #NLTK
tokens = [my_stemmer.stem(token.text) for token in doc]
```



```
['the', 'san', 'francisco', '-', 'base', 'restaur', 'doe', "n't", 'charg',
                                '$', '10', '.']
```

More about stemming:

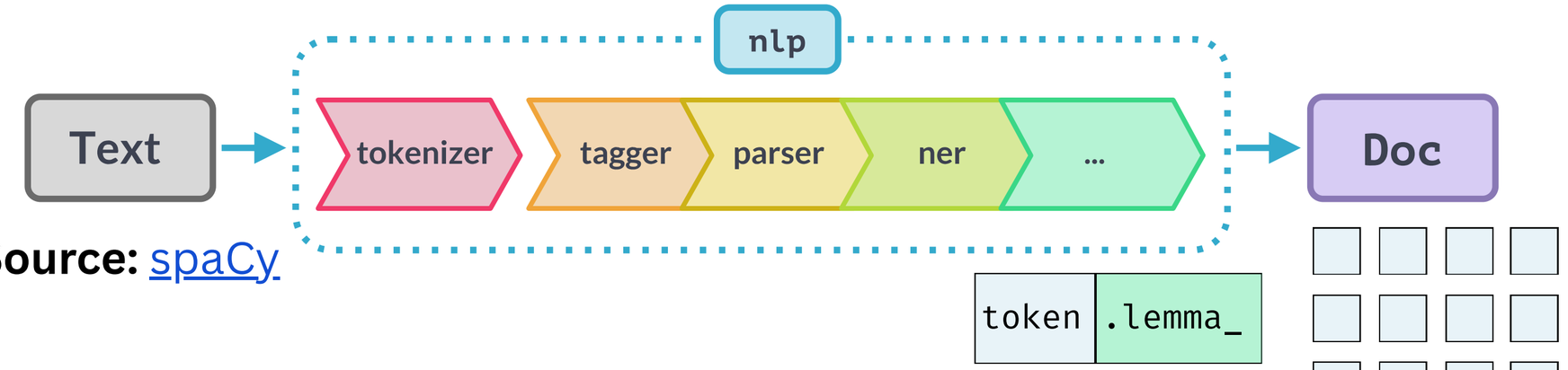
- Multi-language algorithms 🌐
- Multi-language demo! 🤓

Normalization: Lemmatization

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lemma_ for token in doc]
```



Source: spaCy

Normalization: Lemmatization

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lemma_ for token in doc]
```



```
['the', 'San', 'Francisco', '-', 'base', 'restaurant', 'do', 'not',
                        'charge', '$', '10', '.']
```

Normalization: Lemmatization

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
tokens = [token.lemma_ for token in doc]
```









```
['the', 'San', 'Francisco', '-', 'base', 'restaurant', 'do', 'not',
                        'charge', '$', '10', '.']
```

Normalization: Lemmatization

```
"The San Francisco-based restaurant doesn't charge $10."
```

```
tokens = [token.lemma_ for token in doc]
```

```
['the', 'San', 'Francisco', '-', 'base', 'restaurant', 'do', 'not',
                        'charge', '$', '10', '.']
```

text	The	San	Francisco	-	based	restaurant	does	n't	charge	$	10	.
pos_	DET	PROPN	PROPN	PUNCT	VERB	NOUN	AUX	PART	VERB	SYM	NUM	PUNCT
lemma_	the	San	Francisco	-	base	restaurant	do	not	charge	$	10	.

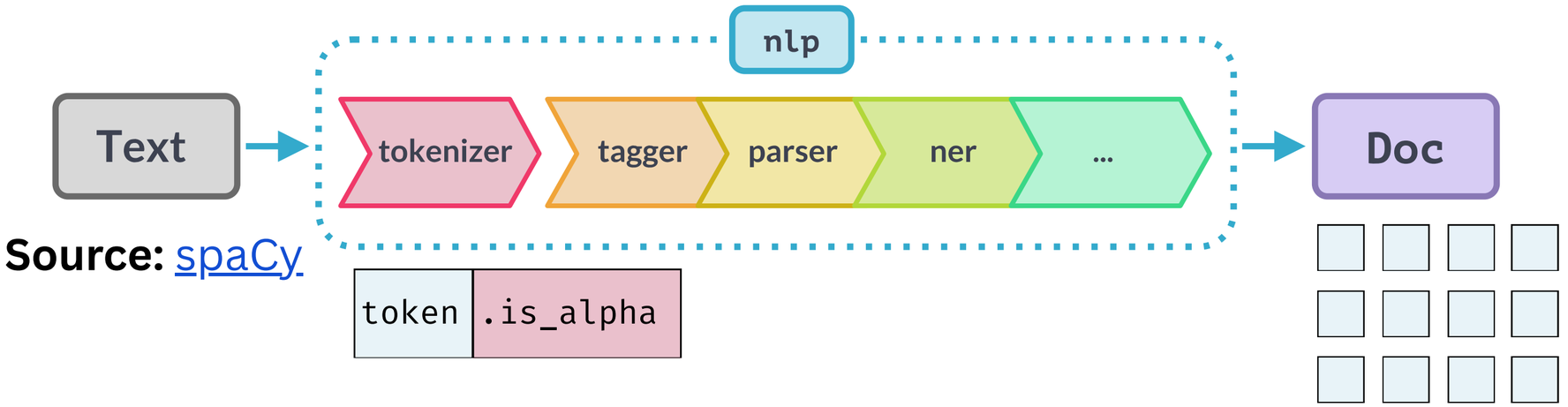
More about lemmatization:

- WordNet: A lexical database for English

Normalization: Remove non-alphabetical

```
"The San Francisco-based restaurant doesn't charge $10."
```

```
[token.text for token in doc if token.is_alpha]
```



Source: spaCy

Normalization: Remove non-alphabetical

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
[token.text for token in doc if token.is_alpha]
```



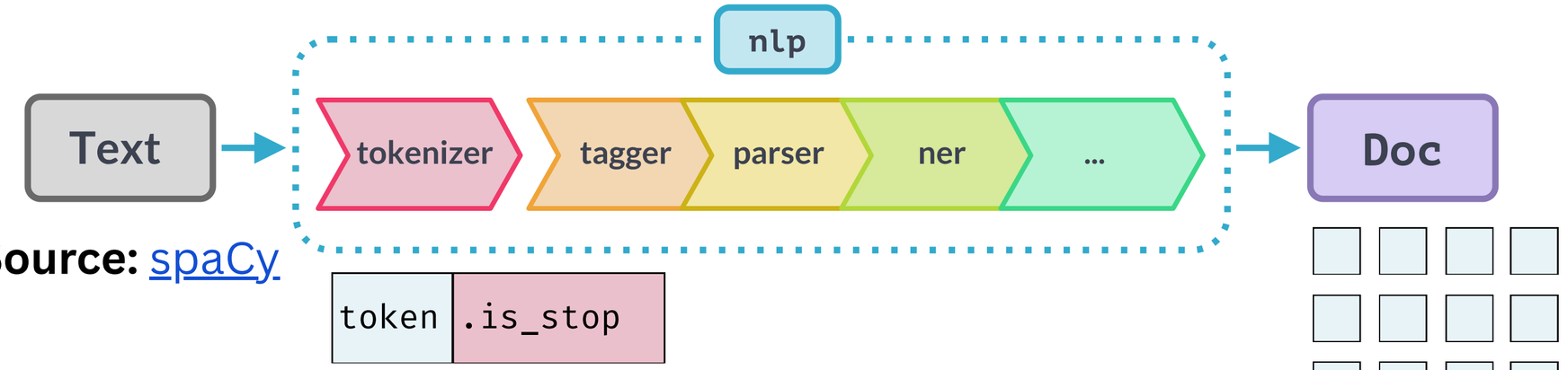
```
['The', 'San', 'Francisco', 'based', 'restaurant', 'does', 'charge']
```

Normalization: Remove stopwords

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
[token.text for token in doc if not token.is_stop]
```



Source: spaCy

Normalization: Remove stopwords

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
[token.text for token in doc if not token.is_stop]
```



```
['San', 'Francisco', '-', 'based', 'restaurant', 'charge', '$', '10', '.']
```

Normalization: Remove stopwords

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
[token.text for token in doc if not token.is_stop]
```



```
['San', 'Francisco', '-', 'based', 'restaurant', 'charge', '$', '10', '.']
```





Normalization: Remove stopwords

```
"The San Francisco-based restaurant doesn't charge $10."
```



```
[token.text for token in doc if not token.is_stop]
```



```
['San', 'Francisco', '-', 'based', 'restaurant', 'charge', '$', '10', '.']
```





SpaCy's stopword lists:

- English
- Find your language's stop_words.py

Normalization: What do we use and when?

"The San Francisco-based restaurant doesn't charge $10."

It turns out this is a good rule of thumb for all languages. Let the data do the talking. Do not stem or lemmatize unless the statistics indicate that it will help your NLP pipeline perform better. Is there not a small amount of meaning that is lost when "smarter" and "smartest" reduce to "smart"? Make sure stemming does not leave your NLP pipeline dumb.

Bottom line, try to avoid stemming, lemmatization, case folding, or synonym substitution, unless you have a limited amount of text with contains usages and capitalizations of the words you are interested in. And with the explosion of NLP datasets, this is rarely the case for English documents, unless your documents use a lot of jargon or are from a very small subfield of science,

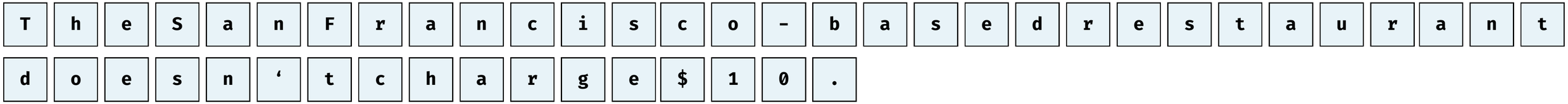
Source: Hobson & Dyshel (2024)

Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10.”

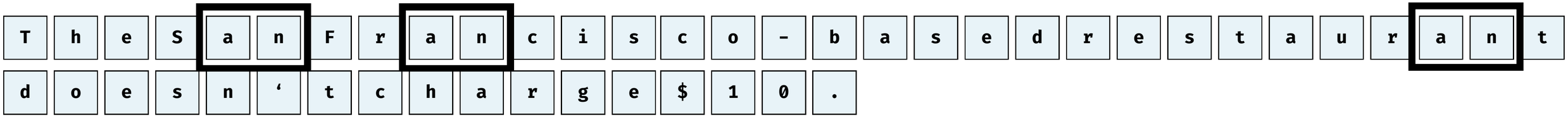
Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10.”



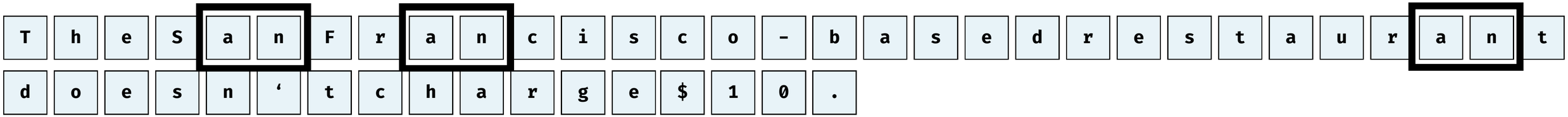
Data-Driven Tokenize: Byte-Pair Encoding

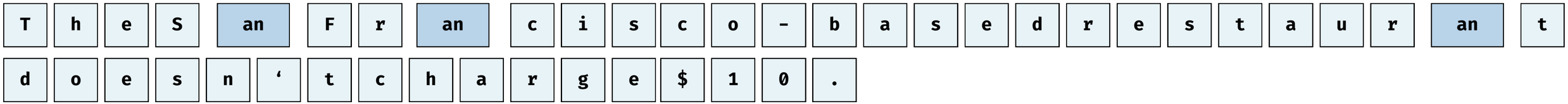
"The San Francisco-based restaurant doesn't charge $10.”



Data-Driven Tokenize: Byte-Pair Encoding

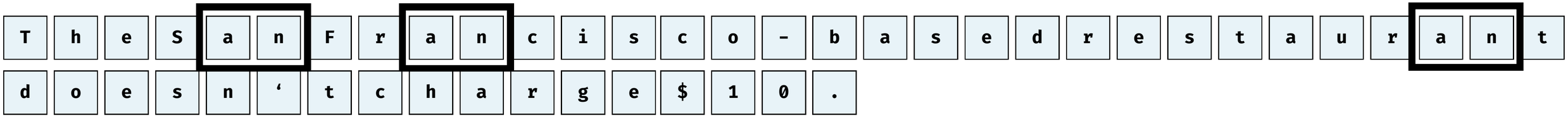
"The San Francisco-based restaurant doesn't charge $10."

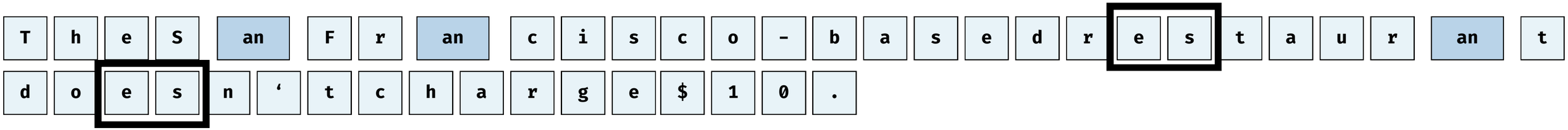




Data-Driven Tokenize: Byte-Pair Encoding

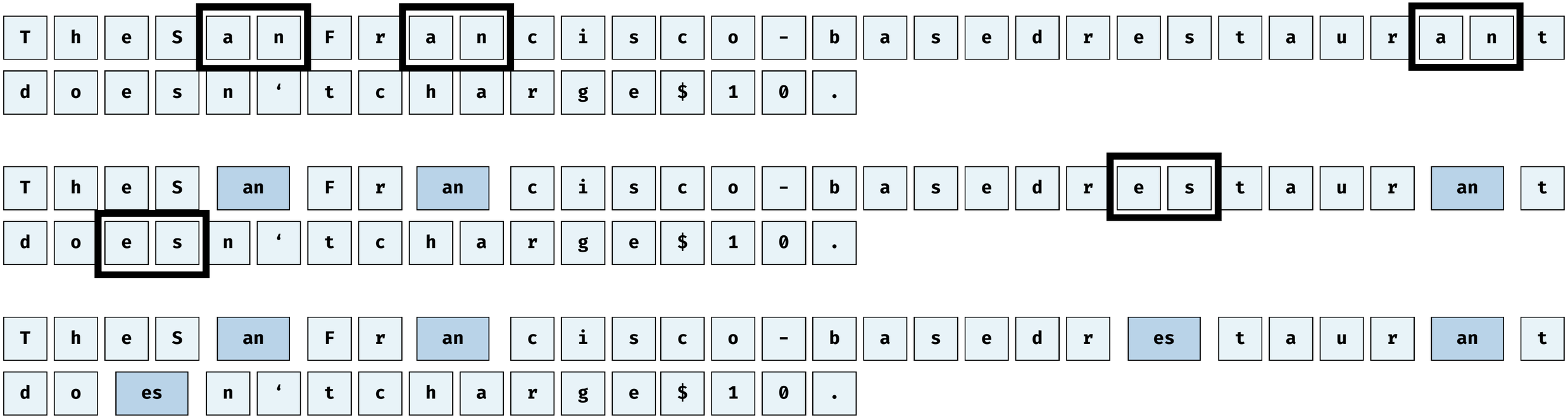
"The San Francisco-based restaurant doesn't charge $10."





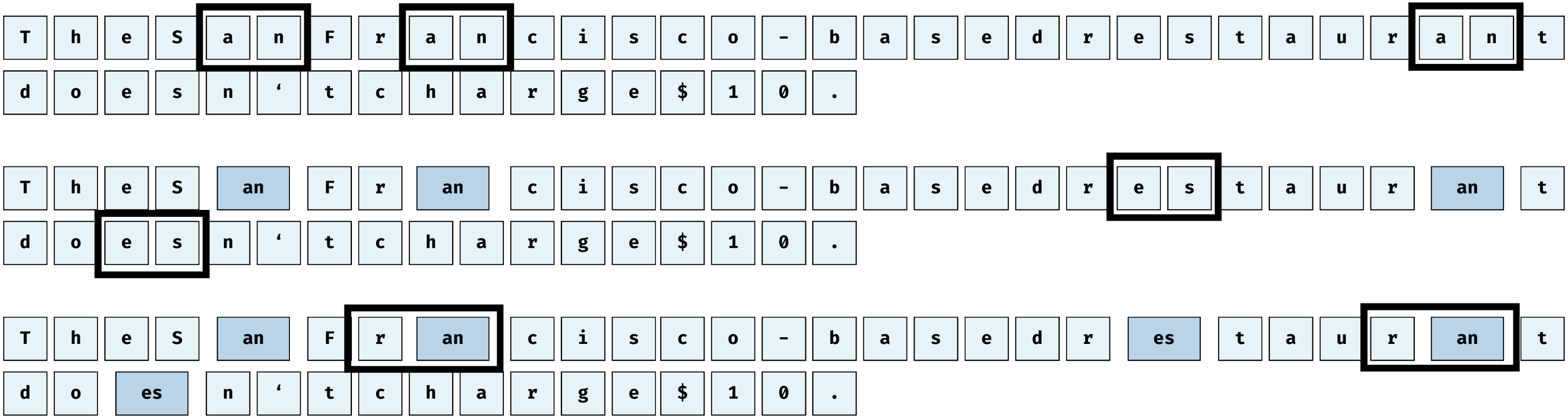
Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10."



Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10."



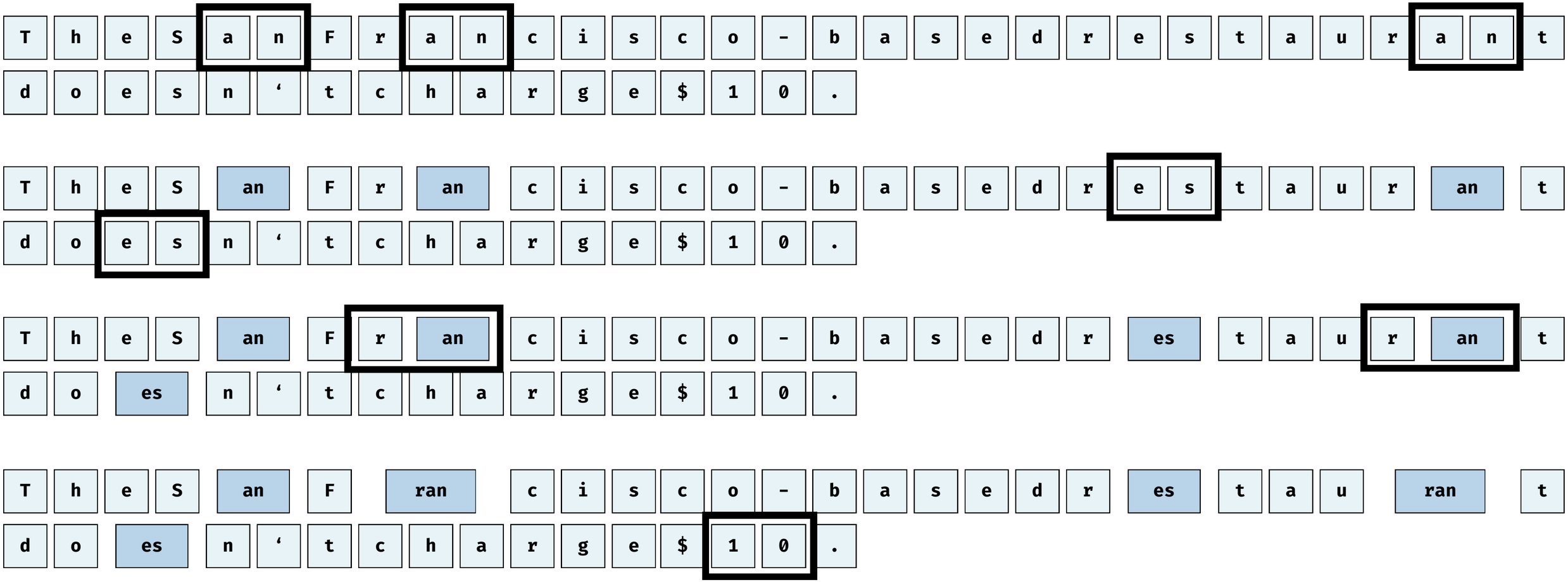
Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10."



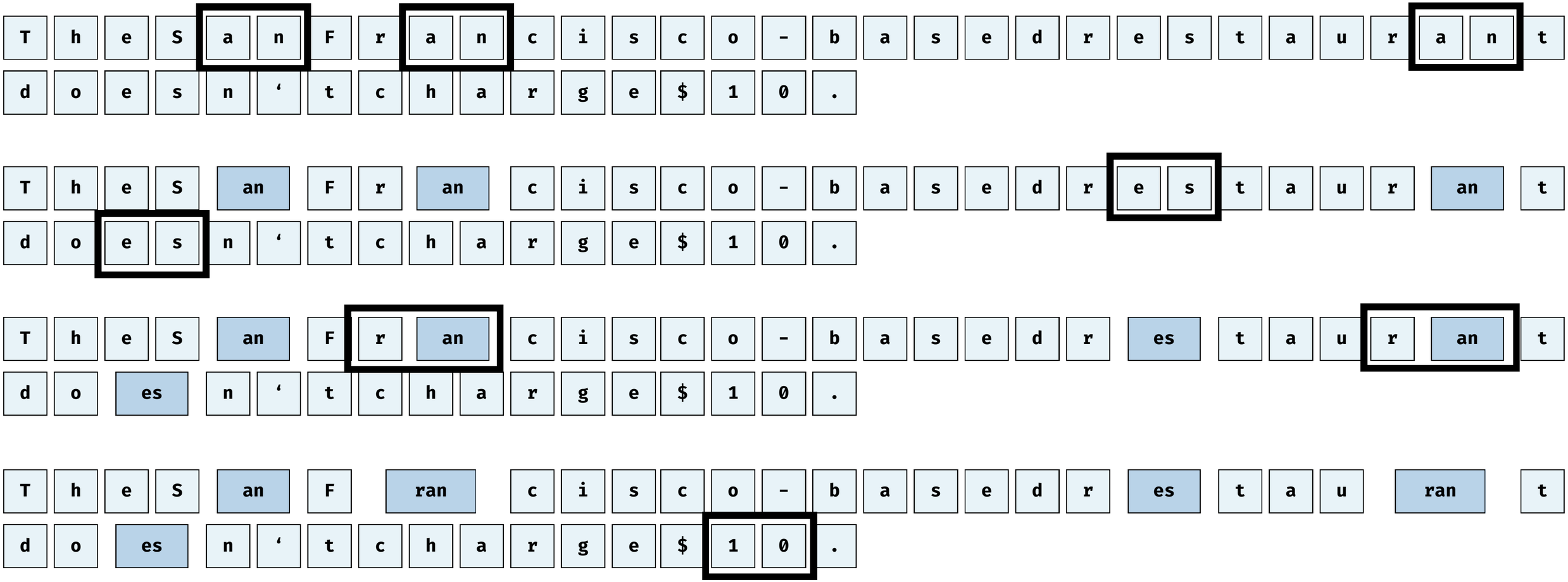
Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10."



Data-Driven Tokenize: Byte-Pair Encoding

"The San Francisco-based restaurant doesn't charge $10."



Data-Driven Tokenize: Byte-Pair Encoding

```
"The San Francisco-based restaurant doesn't charge $10."
```

After 25 merges:

```
['The', 'San', 'Franci', 'sco', '-', 'bas', 'ed', 'res', 'tau', 'rant',
        'do', 'esn', "'", 't', 'ch', 'arge', '$', '10', '.']
```

After 31 merges:

```
['The', 'San', 'Francisco', '-', 'based', 'restaurant', 'doesn', "'", 't',
                    'charge', '$', '10', '.']
```

Data-Driven Tokenize

```
"The San Francisco-based restaurant doesn't charge $10.”
```

More information about:

- Byte-Pair Encoding (bottom-up)
- WordPiece (bottom-up)
- Unigram (top-down)

Next Week: Vectorization! ➡️

